

Technical Project Report: Universal Thrust Bed TinyML Anomaly Detection System

1. Project Overview & Objective

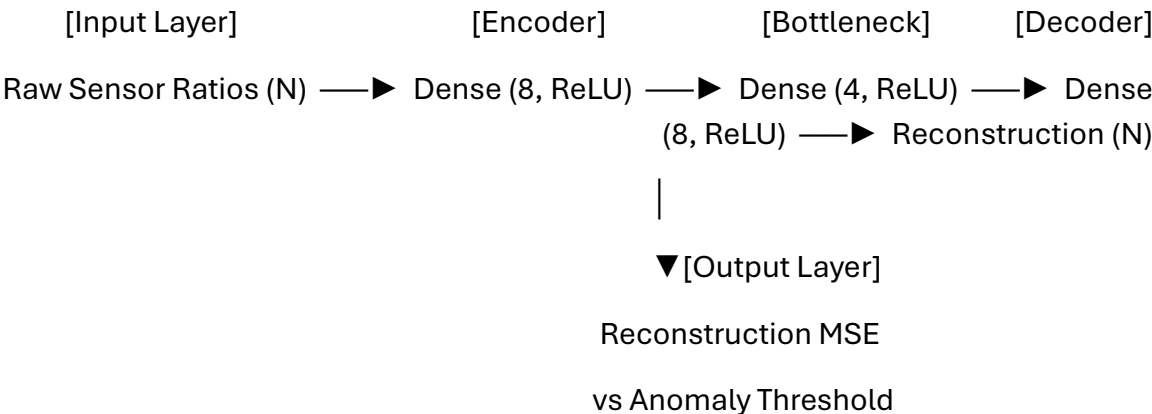
This project presents an end-to-end TinyML Multi-Sensor Anomaly Detection and Edge Telemetry System designed for propulsion and motor thrust test beds (e.g., UAV motors, electric propulsion, robotic actuators, turbines).

Primary Objectives

- Unsupervised Fault Detection:** Automatically detect mechanical degradation, bearing seizure, propeller damage, electrical surges, and motor demagnetization using only normal operational baseline data.
- Ultra-Low-Power Edge Deployment:** Run real-time deep neural network inference directly on resource-constrained microcontrollers (e.g., ARM Cortex-M4/M7, STM32, ESP32) within a < 10 KB RAM tensor arena and sub-millisecond execution latency.
- Interactive Telemetry & Digital Twin Dashboard:** Provide a comprehensive UI for automated model training, hardware-in-the-loop (HIL) serial communication, virtual MCU emulation (via Renode), and interactive stress testing.

2. Machine Learning Model Architecture

The core of the detection pipeline is an Undercomplete Deep Autoencoder Neural Network trained using unsupervised reconstruction loss.



2.1 Model Specifications

Parameter	Value / Specification	Description
Model Type	Deep Feedforward Autoencoder	Unsupervised feature compression & reconstruction
Input Dimension (\$N\$)	Dynamic (4 to 12 features)	Raw sensors + physical ratios + rolling volatilities
Hidden Layers	8 \to 4 (Bottleneck) \to 8 \to N	Symmetric undercomplete architecture
Activation Functions	Hidden: ReLU Output: Linear	Ensures compatibility with standard TFLite Micro kernels
Loss Function	Mean Squared Error (MSE)	$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (X_i - \hat{X}_i)^2$
Optimizer	Adam (\$\beta_1=0.9\$, \$\beta_2=0.999\$)	Stable convergence on time-series telemetry
Model Footprint	~3.0 KB (.tflite quantized)	Fits comfortably in internal Flash memory
RAM Footprint	~10 KB (Tensor Arena)	No dynamic heap allocation (malloc) required

2.2 Why Autoencoder for Anomaly Detection?

In real-world propulsion test beds, nominal data is abundant, but failure data is dangerous, expensive, and rare.

- The autoencoder is trained only on healthy operational cycles.
- It learns the mathematical manifold and physical correlations between sensors (e.g., current proportional to RPM, vibration scaling quadratically with rotational speed).
- When anomalous conditions occur (such as a chipped propeller or stator winding short), the correlations break down. The network fails to reconstruct the corrupted pattern, causing a sharp surge in the Mean Squared Error (MSE) score.

2.3 Threshold Calibration Methods

The system calculates a statistical threshold from the training error distribution:

1. **Robust IQR Fence Rule (Default):** $\text{Threshold} = Q_3 + 1.5 \times (Q_3 - Q_1)$ *Advantage:* Resistant to outliers and noise present during baseline data collection.
2. **Statistical Sigma Rule:** $\text{Threshold} = \mu_{\{\text{train_mse}\}} + k \cdot \sigma_{\{\text{train_mse}\}}$ ($k \in [2.0, 6.0]$)
3. **Percentile Cutoff:** 98th or 99th percentile of nominal reconstruction error.
4. **Dynamic Warmup Baseline:** An adaptive zero-shot rolling baseline computed over the first 50–100 samples of an active run.

3. End-to-End Process Flow

flowchart TD

A [Raw Telemetry Ingestion
Excel / CSV / UART Serial / GUI Simulator] --> B
[Sanitization & Feature Engineering]

B --> C [Scale-Invariant Physical Ratios & Volatility Metrics]

C --> D [StandardScaler Z-Score Normalization]

D --> E [Gaussian Noise Augmentation]

E --> F [Train TinyML Autoencoder Model]

F --> G [Calculate Error Distribution & Auto-Calibrate Threshold]

G --> H [Quantize Model to TFLite Flatbuffer]

H --> I [Generate C Header & Embedded Deployment Package]

subgraph Edge_Inference ["Edge Inference Execution (MCU / Python Engine)"]

J [Incoming Live Telemetry Sample] --> K [Z-Score Scaling with Header Means/Stds]

K --> L [Forward Pass: MicroInterpreter Reconstruction]

L --> M [Compute Reconstruction MSE]

M --> N {MSE > Threshold?}

N -- Yes --> O [FLAG: ANOMALY DETECTED\nFlight Status: NO-GO]

N -- No --> P[FLAG: NOMINAL / HEALTHY\nFlight Status: GO]

end

I -.-> Edge_Inference

Step-by-Step Breakdown:

1. **Data Ingestion:** Reads multi-channel sensor logs from multi-sheet Excel files, CSV recordings, or real-time UART streams.
2. **Domain-Specific Feature Engineering:**
 - **Cross-Sensor Ratios:**
 - $\text{Current-per-RPM} = \frac{I}{\text{RPM}} + \epsilon$ (detects motor back-EMF anomaly / electrical fault).
 - $\text{Vibration-per-RPM}^2 = \frac{G}{\text{RPM}^2} + \epsilon$ (detects mechanical imbalance / bearing wear).
 - $\text{Thrust-per-Current} = \frac{T}{I} + \epsilon$ (evaluates aerodynamic propeller efficiency).
 - **Rolling Volatilities:** Windowed coefficient of variation ($\frac{\sigma_{\text{rolling}}}{\mu_{\text{rolling}}}$) to capture high-frequency instability.
3. **Standardization & Generalization:** Features are normalized via Z-score scaling ($\frac{x - \mu}{\sigma}$). Gaussian noise ($\sigma = 0.05$) is injected during training to prevent overfitting to specific motor sizes.
4. **Auto-Quantization & C Header Generation:** Converts the Keras model to .tflite (int8/float fallback), encodes weights into a static C byte array (model_data.h), and generates model_config.h containing normalization vectors and threshold values.
5. **Real-Time Evaluation:** For every incoming time-slice, the system computes the residual error vector and compares total MSE to the threshold.

4. Input and Output Specifications

4.1 System Inputs

Input Category	Parameters / Fields	Data Type / Unit	Ingestion Method
Physical Sensors	Rotational Speed (rpm)	Integer / Float [RPM]	Ingested via file, serial port, or interactive slider
	Motor Current (current_a)	Float [Amperes, A]	
	Structural Vibration (vibration_g)	Float [G-force, g]	
	Thrust Output (thrust_kg)	Float [Kilograms, kg]	
	Motor Temperature (temperature_c)	Float [Celsius, °C]	
Hyperparameters	Feature strategy, Noise factor, Training epochs, Batch size, Threshold mode	Categorical / Numerical	Configured via GUI sidebar
Edge Hardware Feed	UART Serial Stream (Baud: 115200)	Formatted ASCII / CSV over COM port	Micro USB / FTDI interface

4.2 System Outputs

Output Category	Output Parameter	Format / Values	Significance
Quantitative Score	Reconstruction MSE	Positive Float (e.g., 0.041829)	Quantitative deviation from learned nominal dynamics

Output Category	Output Parameter	Format / Values	Significance
Health Decision	Binary Status	NOMINAL vs ANOMALY DETECTED	Hard decision boundary against calibrated threshold
Operational Readiness	Flight Status Banner	FLIGHT READY: GO (Green) ABORT / INSPECT: NO-GO (Red)	High-level directive for test bed operators / pilots
Root-Cause Attribution	Feature Residual Breakdown	Relative error bar chart per sensor	Pinpoints <i>which</i> physical parameter caused the trigger (e.g., 85% vibration error = bearing fault)
Firmware Deliverables	anomaly_model.tflite	Binary Flatbuffer (~3 KB)	TensorFlow Lite model file
	model_config.h	C Header File	Macro definitions of means, stds, feature order, and threshold
	model_data.h	C Byte Array (g_model)	Directly compiled into MCU Flash memory
	main_tflm.cpp	C++ Source File	Bare-metal firmware template with tensor arena
	mcu_deployment_package.zip	Archive file	Complete bundle ready for STM32CubeIDE / Keil / ESP-IDF

5. Software Tool Capabilities & Dashboard Modules

The tool is implemented as an interactive, dark-mode Streamlit Telemetry & Analytics Dashboard divided into 7 dedicated operational tabs:

1. **Dataset Inspector (Tab 0):** Inspects raw telemetry files, checks statistical distributions, identifies missing rows, and validates sensor schema mapping.
2. **Real-Time Telemetry Stream (Tab 1):** Simulates a live flight/bench run with animated time-series charts, continuous MSE scoring, dynamic anomaly alerting, and a flight readiness banner.
3. **MCU Model Fit & Residuals (Tab 2):** Provides diagnostic residual histograms, individual sensor error tracking, and visual threshold cutoff lines.
4. **MCU Injection & Serial Hardware Bridge (Tab 3):** Connects to a physical STM32 / ESP32 development board over USB-UART (PySerial) to receive live on-device inferences, and exports the complete C firmware deployment zip.
5. **Interactive Sensor Simulator (Tab 4):** Allows manual manipulation of sensor sliders to test scenarios (e.g., sudden current spikes, bearing friction, RPM drop) and observe instantaneous model reaction.
6. **Latency & Profiling Benchmark (Tab 5):** Profiles inference timing (microseconds per sample), throughput (samples/sec), and memory utilization.
7. **Renode Virtual Socket Emulator (Tab 6):** Emulates an ARM Cortex-M4 (STM32F4 Discovery) in software using Renode, enabling complete firmware verification without requiring physical hardware.